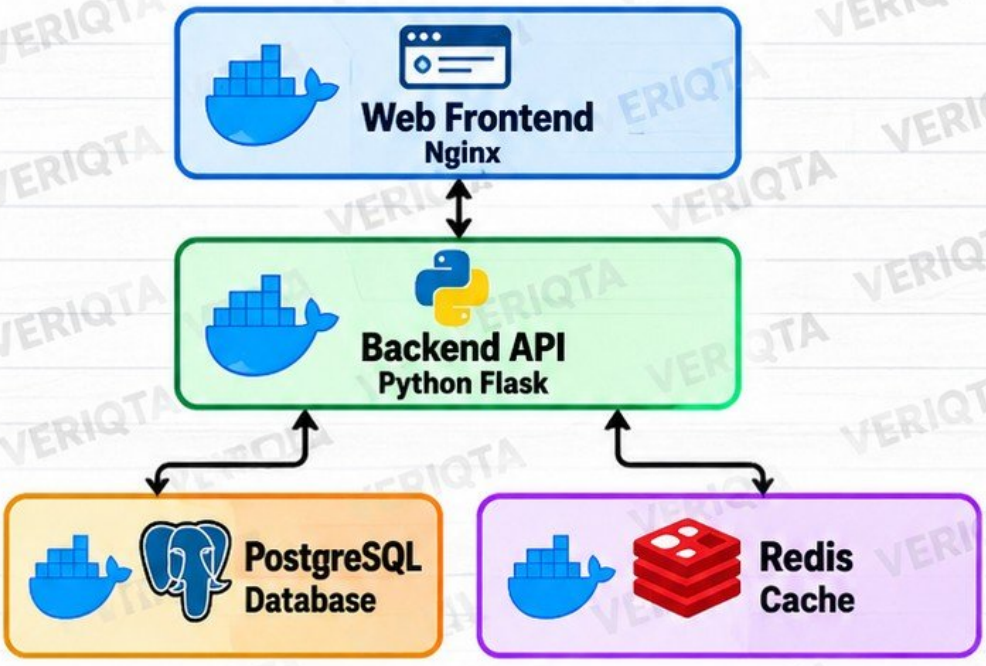
BEGINNER LEVEL

Docker Compose

From Zero to Multi-Container App

- Real Project
- Step by Step
- Practical Examples
- Troubleshooting
- Best Practices
- Production Ready





REAL SETUP | HANDS-ON COMMANDS | COMMON MISTAKES | TROUBLESHOOTING GUIDE | BEST PRACTICES | DEVOPS READY

VERIQTA

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

Docker Compose: From Zero to Multi-Container App

1. Why Docker Compose Exists

1. Real Applications Use Multiple Containers

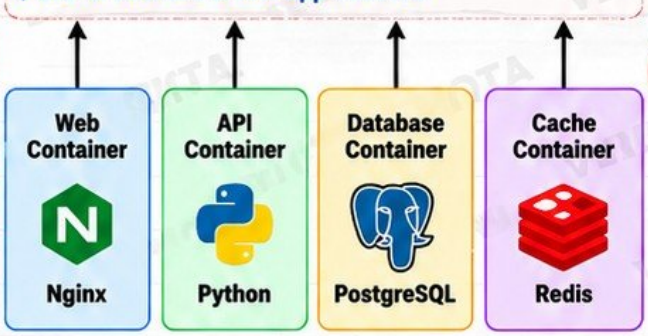
- Modern applications are made of different parts that work together.
- Each part can run in its own container.
- This makes the application flexible, scalable and easier to maintain.
- But starting and managing many containers manually becomes complex.

2. The Problem Without Docker Compose

- You must run multiple docker run commands.
- You must remember ports, networks, volumes and environment variables.
- Starting, stopping and managing containers separately is time-consuming.
- It is easy to make mistakes.
- Recreating the whole application is difficult.

Managing Containers Manually

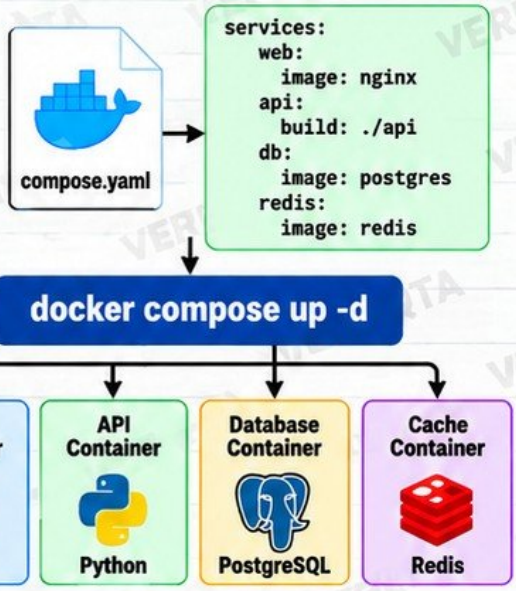
```
$ docker run -d --name web -p 8080:80 nginx
$ docker run -d --name api -p 5000:5000 myapi
$ docker run -d --name db -e POSTGRES_DB=app postgres
$ docker run -d --name redis redis
$ docker network create app-network
```



- ✗ Multiple commands
- ✗ Hard to manage
- ✗ Easy to make mistakes
- ✗ Difficult to recreate

VS

Using Docker Compose



- ✓ One configurations file
- ✓ Starts all services together
- ✓ Easier to manage
- ✓ Easy to recreate and share

3. What Is Docker Compose?

- Docker Compose is a tool for defining and running multi-container applications.
- You describe your services in a compose.yaml file.
- Then you start, stop and manage all services with simple commands.
- It creates a network, sets up volumes, and configures each service for you.

4. Related Docker Tools

Docker Engine
Runs containers on your machine or server.

Dockerfile
Instructions to build an image.

Docker Image
A read-only template used to create containers.

Docker Compose
Manages multiple related containers using a compose.yaml file.

5. Where Engineers Use Compose

- Local development environments
- Testing and staging environments
- Running full application stacks on a single server
- CI environments
- Demo and learning environments
- Small production deployments on a single host

Junior Engineer Insight

Docker Compose does not combing containers into one container. It manages separate services together.

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

VERIQTA

2. Anatomy of a Multi-Container Application

1. VERIQTA Task App Architecture



User Browser
(Your Computer)

1. User accesses the app
http://localhost:8080

Web Frontend
Nginx
Port 8080 (exposed to your computer)

Only this service needs to be exposed to the internet (or your computer).

2. Nginx forwards requests to the API

Backend API
Python Flask
Internal port 5000
(only accessible inside the Compose network)

3. API reads and writes tasks in PostgreSQL

4. API uses Redis for caching

PostgreSQL
Database
Internal port 5432
(only inside network)

Redis
Cache
Internal port 6379
(only inside network)

All services run in the same Docker Compose network and can communicate using service names.

2. What Each Service Does

Web Frontend (Nginx)

- Serves the user interface
- Receives requests from the browser
- Forwards API requests to the backend
- Only service exposed to your computer

Backend API (Python Flask)

- Handles application logic
- Processes API requests
- Reads and writes data in PostgreSQL
- Uses Redis for caching

PostgreSQL (Database)

- Stores application data (tasks)
- Persistent storage using a volume
- Not exposed to the internet
- Accessible only inside the network

Redis (Cache)

- Stores frequently used data in memory
- Helps the app respond faster
- Not exposed to the internet
- Accessible only inside the network

3. Request Flow (Step by Step)

1. User opens the app in a browser.
2. Nginx receives the request and forwards it to the API using the service name.
3. The API reads or writes data in PostgreSQL.
4. The API uses Redis to cache data.
5. The API sends a response back to Nginx.
6. Nginx returns the result to the browser.

4. Why Separate Containers?

- Each service has a specific job.
- Services can be developed, updated and scaled independently.
- You can use the best tool for each job.
- Problems in one service do not bring down the whole application.
- Makes the application easier to maintain.

5. Application Architecture vs Compose Configuration

Application Architecture

- Shows how the services work together.
- Defines the role of each service.
- Describes request flow and dependencies.
- This is a design decision.

Compose Configuration

- A compose.yaml file that creates the services.
- Defines images, ports, networks, volumes, and environment variables.
- This is an implementation of the architecture.

6. Common Mistake

Assuming every container must expose a port to the internet.

- Database and cache usually do not need public access.
- Keep them internal for security.
- Only expose the services that users need to reach (usually the frontend).

Junior Engineer Insight

Docker Compose does not combine containers into one container.
It manages separate services together.

3. Install and Verify Docker Compose

1. What You Need

Docker Engine

- The core platform that containe containers.
- Required for Docke Compose to wor work.
- Install the latest stable version.



Docker Engine
(Required)

Compose Plugin

- Built into the Docker CLI (v2+).
- N separ t loder installation needed on most systems.
- Provides the docker compose command.



Compose Plugin
(Included with Docker)

2. Moderhon Docker Compose Syntax

Use the docker compose command (integrated plugin) instead of the old docker-compose command.

```
# Modern (recommended)
docker compose up -d

# Legacy (older)
docker-compose up -d
```

Junior Engineer Tip
The modern syntax is simpler, better supported and works with Compose V2.

3. Verify Docker Installation

```
docker version
```

Shows the Docker Engine and CLI versions. You should see client and server information.

```
$ docker version
Client: Docker Engine - Community
 Version:  24.0.6
Server: Docker Engine - Community
 Version:  24.0.6
```

4. Verify Docker Compose

```
docker compose version
```

Shows the Compose plugin version. You should see the version number for docker compose.

```
$ docker compose version
Docker Compose version v2.24.5
```

5. Check Engine Connectivity

```
docker cónfo
```

Confirms Docker can talk to the daemon and shows system information.

```
$ docker info
Server Version: 24.0.6
Storage Driver: overlay2
...
Containers: 0
Images: 0
```

6. Diagnose a Cocker Daemon Error

Common error:

> Cannot connect to the Docker daemon at unix:///var/run/docker.sock.
> Is the docker daemon running?

Possible causes:

- Dockerservice is not running.
- You do not have permission to access the socket.
- Docker is not installed correctly.

How to fix:

1. Start the Docker service (systemctl start docker).
2. Add your user to the docker group (if needed).
3. Check the socket permissions.
4. Try the command again.

7. Practice: Confirm It Works

1. Run `docker version`.
2. Run `docker compose version`.
3. Run `docker info`.
4. Create a simple compose.yaml with one service.
5. Run `docker compose up -d`.
6. Verify the service is running with `docker compose ps`.

Quick Test
Expose a simple web service and open it in your browser.

8. Security Reminder

Use Docker access only on a trusted machine.

Docker daemon access is highly privileged.

- It can start, stop and modify containers.
- It can access your files and networks.
- Treat it like root access.

Remember This

Docker Compose is not a replacement for Docker.
It works with Docker to manage multiple containers as one application.

4. Your First compose.yaml File

1. YAML Basics (Simple Structure)

- YAML uses indentation to show structure.
- Each item is written as a key: value pair.
- Indentation must use spaces (not tabs).
- Keep the same indentation level for items that belong together.

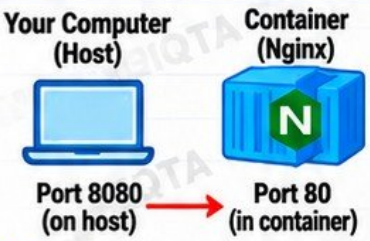
Example:

```
services:
  web:
    image: nginx
```

2. Key Parts of a Service

Key	What It Does
services	Defines the containers in your application.
image	The container image to use.
container_name	Optional custom name. Usually not needed.
ports	Maps a host port to a container port.

3. Host Port vs Container Port



- **Host port:** The port on your computer you access from a browser.
- **Container port:** The port the application listens on inside the container.
- In this example, port 8080 on your computer is forwarded to port 80 in the Nginx container.

4. Create Your First compose.yaml

```
1  services:
2    web:
3      image: nginx
4      container_name: veriqta-nginx
5      ports:
6        - "8080:80"
```

Explanation:

1. **services:** Defines the services in this application.
2. **web:** Name of the service.
3. **image: nginx:** Uses the official Nginx image.
4. **container_name:** Custom name (optional).
5. **ports:** Maps port 8080 on your computer to port 80 in the container.

5. Validate the Configuration

`docker compose config`

- Checks if your compose.yaml file is correct.
- Shows the final configuration Docker will use.
- Helps you catch indentation or syntax errors.

```
$ docker compose config
name: veriqta-app
services:
  web:
    image: nginx
    container_name: veriqta-nginx
    ports:
      - target: 80
        published: "8080"
        protocol: tcp
        mode: ingress
```

6. Start the Service

`docker compose up -d`

- Starts the container in detached mode (runs in the background).
- Downloads the image if needed.
- Creates and starts the service.

```
$ docker compose up -d
[+] Running 2/2
 ✓ web Pulled
 ✓ Container veriqta-nginx
   Started
```

7. Test in Your Browser

`Open http://localhost:8080`

- You should see the Nginx welcome page.
- This confirms Docker Compose is working.

8. View Running Services

`docker compose ps`

- Shows the status and ports of your services.

```
$ docker compose ps
NAME           IMAGE   PORTS                  STATUS
veriqta-nginx  nginx   0.0.0.0:8080->80/tcp   Up
```

9. Common Failures

Invalid YAML Indentation

```
services:
  web:
   image: nginx
```

Error: services and web must be indented correctly using spaces.

Host port already in use

Error: Bind for 0.0.0.0:8080 failed: port is already allocated.

Fix:

- Use a different host port, e.g. "8081:80".
- Stop the application using the port.

10. Junior Engineer Insight

Docker Compose does not combine containers into one container. It manages separate services together.

11. Security Reminder

Use Docker access only on a trusted machine. Docker daemon access is highly privileged and can manage your entire system.

5. From Dockerfile to Compose Service

1. Why Build an Application Image?

- Real applications contain your own code and dependencies.
- You cannot run your application directly without an image.
- A Dockerfile defines how to package your application.
- Docker Compose can build this image automatically and run it as a service.

2. build vs image

build

- Builds an image from a Dockerfile.
- You provide the source code and build instructions.
- Used for your own application code.
- Example: `build: .`

image

- Uses an existing image from a registry (e.g. Docker Hub).
- You only specify the image name and tag.
- Used for ready-made applications.
- Example: `image: nginx`

3. Project Structure

```
veriqta-task-app/
├── app/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
└── compose.yaml
```

4. Minimal Python Flask Backend

app/app.py

```python
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/')
def home():
    return jsonify({"message":
           "Hello from VERIQTA API!"})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

5. Dockerfile for the Backend

app/Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir
    -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

6. Requirements File

app/requirements.txt

```
flask==3.0.0
```

7. Composs Service for the API

compose.yaml

```yaml
services:
  api:
    build: ./app
    container_name: veriqta-api
    ports:
      - "5000:5000"
```

8. How It Works: From Code to Running Container

Source Code → app.py, requirements.txt, Dockerfile

→ **Dockerfile** → Instructions to build the image

→ **Docker Image** → Built from your code

→ **Container** → Runs your application on port 5000

9. Build and Start with Compose

```
docker compose up --build
```

- Builds the image if needed.
- Starts the container in detached mode.
- Uses the compose.yaml configuration.
- Shows build steps and starts the service.

10. When Does Compose Rebuild the Image?

Compose rebuilds when:

- You run `docker compose up --build`.
- The Dockerfile changes.
- Files copied into the image change (e.g. requirements.txt).
- You use `--no-cache`.

Compose reuses the existing image when:

- You run `docker compose up -d` (without `--build`).
- No relevant files have changed.
- The image already exists and is up to date.
- This makes container startup faster.

11. Common Beginner Mistake

Assuming every code change automatically rebuilds the image.

- Docker does not automatically rebuild when you edit your code.
- You must run `docker compose up --build` or use a development workflow such as bind mounts or watch mode.

12. Practice: Verify It Works

1. Run `docker compose up --build`.
2. Check the running service: `docker compose ps`.
3. Open your browser: http://localhost:5000
4. You should see: {"message": "Hello from VERIQTA API!"}

13. Key Takeaways

- Build your own application image using a Dockerfile.
- Use `build:` to build the image in Compose.
- Use `image:` for ready-made images.
- `docker compose up --build` builds and starts the service.
- Compose only rebuilds when needed.

14. Security Reminder

Use Docker access only on a trusted machine. Docker daemon access is highly privileged and can manage your entire system.

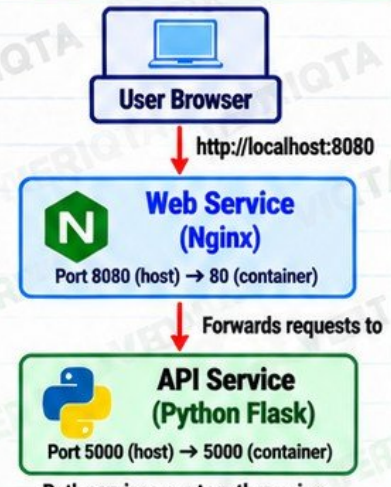
6. Running Multiple Services Together

1. Web and API Services in compose.yaml

compose.yaml

```yaml
services:
  web:
    image: nginx
    container_name: veriqta-web
    ports:
      - "8080:80"
    depends_on:
      - api
  api:
    build: ./app
    container_name: veriqta-api
    ports:
      - "5000:5000"
```

2. Application Architecture



Both services run together using Docker Compose and can communicate using service names.

3. How Compose Creates a Project

- When you run `docker compose`, Compose creates a project.
- The project name is the folder name (or you can set it with `--project-name`).
- All resources (containers, network, volumes) are grouped under this project.
- Services can communicate using the service names (web, api).

Check the project name:

```
$ docker compose ls
NAME              STATUS     CONFIG FILES
veriqta-task-app  running    compose.yaml
```

4. Start Both Services

`docker compose up -d`

- Builds images if needed.
- Creates and starts both containers in detached mode.
- Shows a summary of what is running.

```
$ docker compose up -d
[+] Running 2/2
 ✓ Container veriqta-api   Started
 ✓ Container veriqta-web   Started
```

5. Inspect Running Services

`docker compose ps`

- Shows the status, ports and names of your services.
- Helps you confirm both web and api are running.

```
$ docker compose ps
NAME         IMAGE        PORTS                    STATUS
veriqta-web  nginx        0.0.0.0:8080->80/tcp     Up
veriqta-api  veriqta-api  0.0.0.0:5000->5000/tcp   Up
```

6. Stop the Services

`docker compose stop`

- Stops the running containers.
- Containers are not removed.
- You can start them again quickly with `docker compose start`.

```
$ docker compose stop
 ✓ Container veriqta-web   Stopped
 ✓ Container veriqta-api   Stopped
```

7. Stop vs Remove Containers

docker compose stop

- Stops the running containers.
- Containers still exist (preserved).
- You can start them again with `docker compose start`.
- Useful for temporary shutdown.

docker compose down

- Stops and removes containers.
- Also removes project network. (by default).
- Use `-v` to also remove volumes.
- You will need to run `docker compose up` again to recreate everything.

8. Restart a Specific Service

`docker compose restart api`

- Restarts only the API service.
- Useful after making code changes or configuration updates.

```
$ docker compose restart api
 ✓ Container veriqta-api   Restarting
 ✓ Container veriqta-api   Started
```

9. Practice: Your Turn

1. Run `docker compose up -d`.
2. Check the running services with `docker compose ps`.
3. Open http://localhost:8080 (you should see Nginx).
4. Stop the services with `docker compose stop`.
5. Start them again with `docker compose start`.
6. Restart the API service with `docker compose restart api`.

10. Production Awareness

- A running container is not necessarily a functioning application.
- Always verify that your application is working (check logs, test endpoints).
- Use health checks in real environments.

11. Common Issues

Port already in use

Error: Bind for 0.0.0.0:8080 failed: port is already allocated.

Fix:

- Use a different host port (e.g. 8081:80).
- Stop the process using the port.

Service not starting

Check logs:

`docker compose logs api`

- Look for syntax errors, missing files or build failures.
- Fix the issue and try again.

12. Key Takeaways

- Define all services in one compose.yaml file.
- Use `docker compose up -d` to start multiple services.
- Use `docker compose ps` to inspect their state.
- Use `docker compose stop` or `down` as needed.
- Always verify your application is actually working.

7. Docker Compose Networking Explained

1. What Is the Default Compose Network?

- When you run docker compose up, Compose creates a default network for your application.
- All services in the same Compose file join this network automatically.
- Services can communicate with each other using service names.
- You do not need to create the network manually for simple setups.

```
$ docker compose up -d
$ docker network ls

NAME                      DRIVER    SCOPE
veriqta-taskapp_default   bridge    local
```

2. Service Name DNS Resolution

- Each service gets a DNS name equal to its service name.
- You can reach the api service from the web service using http://api:5000.
- Docker Compose handles the DNS resolution automatically.
- The service name is more stable than a container IP address.

Example:

```
http://api:5000

# api is the service name
# 5000 is the container port
# Used inside the Compose network
```

3. Host Port vs Container Port

- Container port: The port the application listens on inside the container.
- Host port: The port on your computer that you access from your browser.
- You only publish ports that need to be accessed from outside the Compose network.

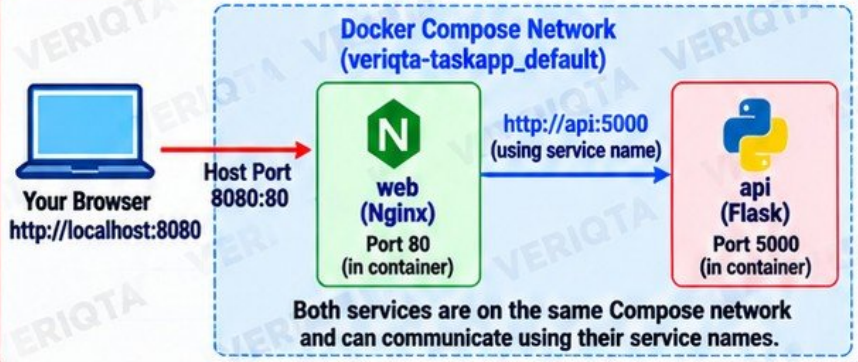
Example (web service):

```
ports:
  - "8080:80"
```

Container port (inside container)

Host port (on your computer)

4. Web → API Communication (Diagram)



5. Inside Container vs Host Machine

localhost inside a container refers to the container itself.



localhost on your computer refers to your host machine.



http://localhost:8080
(accesses the web service through the published port)

6. Demonstration: Web Reaches API by Service Name

1. Test from inside the web container

```
$ docker compose exec web sh
# Install curl (if not available)
  apt-get update && apt-get install -y curl
# Call the API using the service name
  curl http://api:5000
```

Expected output:

{"message":"Hello from VERIQTA API!"}

2. Test from your browser

Open your browser and go to:

http://localhost:8080

You should see the Nginx welcome page from the web service.

> **Welcome to nginx!**
>
> If you see this page, the web service is working.

7. Why Not Use Container IP Addresses?

- Container IP addresses can change whenever containers are recreated.
- They are not stable.
- Docker Compose provides automatic service-name DNS resolution.
- Always use service names (web, api) instead of IP addresses.

8. Common Failure: Using localhost to Reach Another Container

Problem:
You are inside the web container and try to reach the API using localhost:5000.

```
$ docker compose exec web sh
$ curl http://localhost:5000
# This will fail because it tries to reach port 5000
# on the web container itself, not the api container.
```

9. Correct Way

Use the service name instead.

```
$ docker compose exec web sh
$ curl http://api:5000
```

10. Key Takeaways

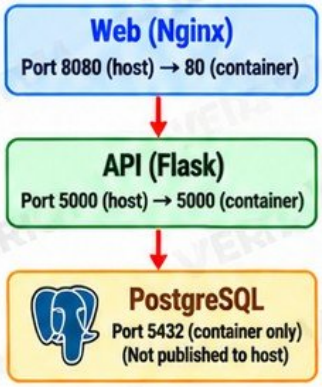
- Compose creates a default network.
- Services can reach each other using service names.
- localhost inside a container refers to the container itself.
- Only publish host ports that you need.
- Do not hardcode container IP addresses.

8. Adding PostgreSQL to the Application

1. Why Add a Database?

- Real applications need a database to store data permanently.
- Here we add PostgreSQL as a separate service managed by Docker Compose.
- The API (Flask) will connect to PostgreSQL using the service name.
- The database runs in its own container on the same Compose network.

2. Application Architecture



3. PostgreSQL Service in compose.yaml

compose.yaml

```yaml
services:
  db:
    image: postgres:16
    container_name: veriqta-db
    environment:
      POSTGRES_DB: veriqta
      POSTGRES_USER: veriqta
      POSTGRES_PASSWORD: veriqta123
    ports:
      # Not publishing to host
      # - "5432:5432"
    volumes:
      - db_data:/var/lib/postgresql/data
networks:
  - default
```

4. Understanding the Environment Variables

Variable	What It Does	Example
POSTGRES_DB	Name of the database to create.	veriqta
POSTGRES_USER	Username to access the database.	veriqta
POSTGRES_PASSWORD	Password for the user.	veriqta123

5. Connect Flask to PostgreSQL

app/app.py

```python
from flask import Flask
import os
import psycopg2

app = Flask(__name__)

DB_HOST = "db"      # service name
DB_NAME = "veriqta"
DB_USER = "veriqta"
DB_PASSWORD = "veriqta123"
DB_PORT = "5432"

conn = psycopg2.connect(
    host=DB_HOST,
    dbname=DB_NAME,
    user=DB_USER,
    password=DB_PASSWORD,
    port=DB_PORT
)
```

6. Why Not Publish the DB Port?

- The database only needs to be accessed by the API inside the Compose network.
- Publishing the port to your host is not required and can be a security risk.
- Keeping the port internal prevents external access from the internet.
- Services in the same Compose network can reach the database using the service name (db) and port 5432.

✓ API connects to: db:5432 (inside the network)

7. Start the Stack

```
docker compose up -d
```

- Downloads the PostgreSQL image.
- Creates the database using the environment variables.
- Starts all services (web, api, db).
- Runs in detached mode.

8. Verify the Services are Running

```
docker compose ps
```

```
NAME          IMAGE          PORTS                    STATUS
veriqta-web   nginx          0.0.0.0:8080->80/tcp     Up
veriqta-api   veriqta-api    0.0.0.0:5000->5000/tcp   Up
veriqta-db    postgres:16    5432/tcp                 Up
```

9. Test the Database Connection

```
docker compose exec api python
```

```
>>> import psycopg2
>>> conn = psycopg2.connect(
...     host="db",
...     dbname="veriqta",
...     user="veriqta",
...     password="veriqta123",
... )
>>> print("Connected!")
Connected!
```

10. Create a Task Through the API

Example request:

```
curl -X POST http://localhost:5000/tasks \
  -H "Content-Type: application/json" \
  -d "{"title": "Learn Docker", "description": "Use PostgreSQL"}"
```

Then check in the database:

```
docker compose exec db psql -U veriqta -d veriqta
```

```
SELECT * FROM tasks;
 id |    title     |     description
----+--------------+---------------------
  1 | Learn Docker | Use PostgreSQL
```

11. Common Database Connection Failures

- **Could not translate host name "db"**
  Cause: Wrong service name.
  Fix: Use the correct service name (db).
- **Connection refused**
  Cause: Database not ready yet.
  Fix: Add depends_on and/or retry logic.
- **Authentication failed**
  Cause: Wrong username or password.
  Fix: Check POSTGRES_USER and POSTGRES_PASSWORD.
- **Database does not exist**
  Cause: Wrong POSTGRES_DB value.
  Fix: Use the correct database name.

12. Security Reminder

- Never commit real database credentials to Git.
- Use environment variables or a .env file.
- Keep sensitive values out of public repositories.
- Use different credentials for development and production.

13. Key Takeaways

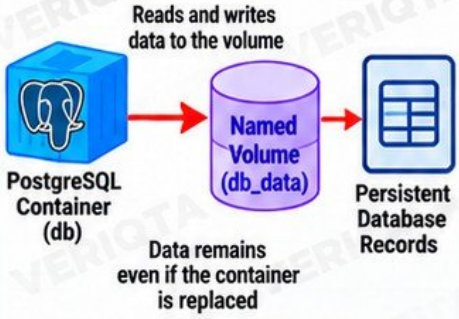
- PostgreSQL runs as a separate service.
- Use environment variables to configure it.
- Connect using the service name (db).
- You do not need to publish the database port.
- Verify the data is stored in the database.
- Keep credentials secure.

9. Docker Volumes and Persistent Data

1. Why Use Volumes?

- Container writable-layer data is not reliable for long-term storage.
- When a container is removed, its data is lost.
- Volumes keep your application data outside the container so it survives container replacement.
- This is important for databases, uploads and any data you want to keep.

2. How a Volume Works (Diagram)



3. Named Volume vs Bind Mount

Feature	Named Volume	Bind Mount
Managed by Docker	Yes	No
Stored location	Docker storage directory	Host machine directory
Best for	Databases, production data	Development, source code
Portability	Easier to move	Depends on host path
Example syntax	volumes: - db_data:/var/lib/postgresql/data	volumes: - ./data:/app/data

4. PostgreSQL with a Named Volume

compose.yaml

```yaml
services:
  db:
    image: postgres:16
    container_name: veriqta-db
    environment:
      POSTGRES_DB: veriqta
      POSTGRES_USER: veriqta
      POSTGRES_PASSWORD: veriqta123
    volumes:
      - db_data:/var/lib/postgresql/data
    ports:
      - "5432:5432" # optional (only if you
                    # need to access from host)
volumes:
  db_data:
```

5. Understanding the Volumes Section

- volumes: defines named volumes used by services.
- db_data: is the name of the volume.
- It is mounted to /var/lib/postgresql/data inside the PostgreSQL container.
- Docker stores the volume data outside the container.
- The data remains even if the container is stopped, removed or replaced.

6. Start the Database

```
docker compose up -d
```

- Creates the named volume (db_data).
- Starts the PostgreSQL container.
- The database stores its data in the named volume.

7. Verify the Volume is Created

```
docker volume ls
```

```
DRIVER    VOLUME NAME
local     veriqta_db_data
local     veriqta_taskapp_db_data
```

8. Demonstrate Persistent Data

1. Start the stack: `docker compose up -d`
2. Create a record in the database:

```
docker compose exec db psql -U veriqta -d veriqta
INSERT INTO tasks (title) VALUES ('Learn Docker');
SELECT * FROM tasks;
```

3. Replace the container:

```
docker compose up -d --force-recreate db
```

4. Check the data again:

```
docker compose exec db psql -U veriqta -d veriqta
SELECT * FROM tasks;
```

- The record is still there because the data is stored in the named volume.

9. What docker compose down Does

```
docker compose down
```

- Stops and removes the containers.
- Removes the network.
- Keeps named volumes (data is preserved).

```
docker compose down -v
```

- Stops and removes the containers.
- Removes the network.
- Removes all named volumes.
- This will delete your database data.

10. Inspect a Volume

```
docker volume inspect db_data
```

```
[
  {
    "Name": "db_data",
    "Driver": "local",
    "Mountpoint": "/var/lib/docker/
volumes/db_data/_data",
    "Scope": "local"
  }
]
```

11. Critical Warning

- Removing a database volume can permanently delete your data.
- A volume is not a backup.
- Always make regular backups of important data.
- Be very careful when using docker compose down -v.

12. Common Mistakes

- Storing database data in the container writable layer.
- Using bind mounts for production databases.
- Running docker compose down -v by accident.
- Assuming the data is safe without a backup.

13. Key Takeaways

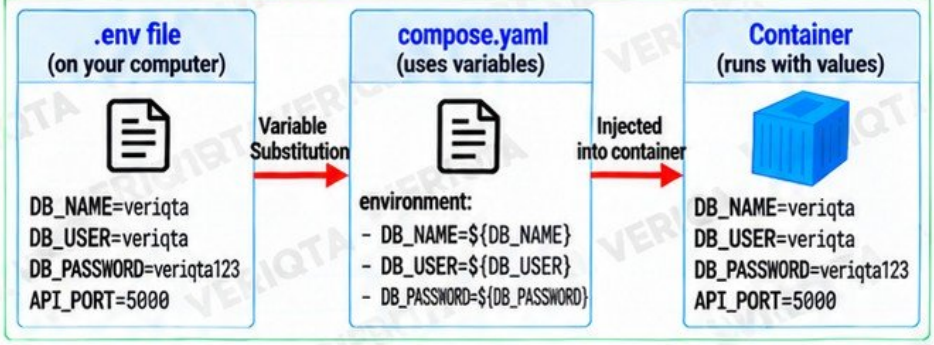
- Use volumes to keep application data.
- Named volumes are ideal for databases.
- Data remains even if containers are replaced.
- docker compose down -v removes volumes.
- A volume is not a backup.
- Always have a backup strategy.

10. Environment Variables and Configuration

1. What Are Environment Variables?

- Environment variables store configuration values.
- They separate configuration from application code.
- You can change configuration without rebuilding your image.
- Used for database credentials, API settings, ports and feature flags.

2. How It Works (Diagram)



3. The .env File (Example)

.env

```
# Application settings
API_PORT=5000

# Database settings
DB_NAME=veriqta
DB_USER=veriqta
DB_PASSWORD=veriqta123
DB_HOST=db
DB_PORT=5432
```

4. Using Variables in compose.yaml

compose.yaml

```
services:
  api:
    build: ./app
    container_name: veriqta-api
    ports:
      - "${API_PORT}:5000"
    environment:
      - DB_HOST=${DB_HOST}
      - DB_NAME=${DB_NAME}
      - DB_USER=${DB_USER}
      - DB_PASSWORD=${DB_PASSWORD}
      - DB_PORT=${DB_PORT}
    depends_on:
      - db
```

5. Compose Interpolation vs Container Environment

Compose Interpolation (on your machine)	Container Environment (inside the container)
Replaces ${VAR} in compose.yaml with values from .env	Passes the values to the container as environment variables
Happens before containers are created	Available to your application at runtime
Used for compose configuration (ports, image names, etc.)	Used by your application (e.g. database connection settings)

6. Database Configuration Example

app/app.py

```
import os
import psycopg2

conn = psycopg2.connect(
    host=os.getenv("DB_HOST", "db"),
    dbname=os.getenv("DB_NAME"),
    user=os.getenv("DB_USER"),
    password=os.getenv("DB_PASSWORD"),
    port=os.getenv("DB_PORT", "5432")
)
```

7. .env.example for Safe Sharing

.env.example

```
# Copy this file to .env and
# update with your own values

API_PORT=5000
DB_NAME=your_db_name
DB_USER=your_db_user
DB_PASSWORD=your_strong_password
DB_HOST=db
DB_PORT=5432
```

8. Practice: Change a Configuration Value

1. Open .env and change a non-sensitive value, for example API_PORT=8000.
2. Recreate the affected service:
   `docker compose up -d api`
3. Check the service is running with the new port:
   `docker compose ps`
4. Open http://localhost:8000 to confirm it works.

9. Why Environment Variables Are Not Automatically Secure

- Values in .env are stored as plain text files on your computer.
- They can be seen by anyone with access to the project files.
- Environment variables help with configuration management, but they are not encrypted or a secret management solution.
- For real secrets in production, use a secret management tool (e.g. Docker Secrets, AWS Secrets Manager, HashiCorp Vault).

10. Common Beginner Mistakes

- ✗ Assuming a .env file automatically makes passwords secret.
- ✗ Committing real credentials to Git.
- ✗ Using the wrong variable name (e.g. DB_PASSWORD vs DB_PASS).
- ✗ Forgetting to recreate the service after changing .env values.
- ✗ Expecting changes in .env to apply automatically without restarting.

11. Key Takeaways

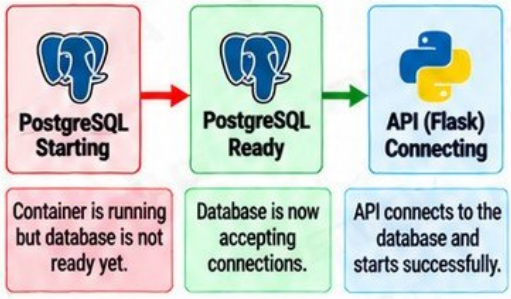
- ✓ Use environment variables to separate configuration from code.
- ✓ Store values in a .env file.
- ✓ Use ${VAR} in compose.yaml.
- ✓ Understand the difference between interpolation and container environment variables.
- ✓ Use .env.example for safe sharing.
- ✓ Environment variables are not automatically secure.

11. Container Startup Order and depends_on

1. Why Startup Order Matters?

- Some services depend on others to work properly.
- Starting a container first does not mean it is ready to accept connections.
- Databases need time to initialize (create files, run migrations, start accepting connections).
- If the application starts too early, it can fail to connect and crash.

2. Startup Process (Diagram)



3. What depends_on Does

- Specifies startup dependencies between services.
- Ensures Docker starts the dependency service first.
- It does not wait for the service to be ready unless you use `condition: service_healthy`.
- Application code should still handle connection retries.

4. Basic depends_on (Starts First)

compose.yaml

```yaml
services:
  api:
    build: ./app
    depends_on:
      - db

  db:
    image: postgres:16
    container_name: veriqta-db
```

- Docker starts db before api.
- This does not wait for the database to be ready.
- The API may still fail if the database is still initializing.

5. Using condition: service_healthy

compose.yaml

```yaml
services:
  api:
    build: ./app
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:16
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U
        ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10
```

- Docker waits until the database passes its health check before starting the API.
- This reduces startup race conditions.
- Your application should still include connection retries.

6. Health Check Explained

- Health check tests if the database is ready to accept connections.
- pg_isready checks if PostgreSQL is running and responsive.
- `interval`: time between checks.
- `timeout`: how long to wait for a response.
- `retries`: number of attempts before considering it unhealthy.

7. Running vs Ready

State	What It Means
Container running	Process is started but database may still be initializing.
Container healthy	Database is ready to accept connections.
Application connected	API can read and write to the database.

8. Example Startup Logs

```
$ docker compose up
veriqta-db   |  Starting PostgreSQL 16...
veriqta-db   |  Database system is ready
               to accept connections
veriqta-db   |  database system is ready
veriqta-api  |  Waiting for database...
veriqta-api  |  Connected to PostgreSQL
veriqta-api  |  Starting Flask application...
veriqta-api  |  Running on http://0.0.0.0:5000
```

9. Common Failure (API Starts Too Early)

```
veriqta-api  |  Error: could not connect
               to server: Connection refused
veriqta-api  |  Is the server running
               on host "db" (172.20.0.3)
               and accepting TCP/IP
               connections on port 5432?
veriqta-api  |  [Errno 111] Connection
               refused
```

This happens when the API starts before PostgreSQL is ready.

10. Practice: Observe Startup Behavior

1. Run the stack: `docker compose up -d`
2. Check the logs: `docker compose logs -f`
3. Observe that the database starts first, then becomes healthy, and the API connects successfully.
4. Try removing the health check and see the difference.

11. Why Applications Need Retries

- Even with `condition: service_healthy`, the application may start before the database is fully ready.
- Network delays and initialization tasks can still cause temporary failures.
- Applications should implement connection retries with exponential backoff.

12. Key Takeaways

- depends_on controls startup order.
- Use `condition: service_healthy` to wait for readiness.
- A running container is not the same as a ready service.
- Applications should still include connection retries.
- Always verify startup behavior in your logs.

13. Beginner Mistakes

- Assuming depends_on waits for the service to be ready.
- Not using a health check for the database.
- Not implementing connection retries in the application.
- Expecting the database to be ready immediately.
- Ignoring startup logs when troubleshooting.

12. Health Checks: Is Your Application Actually Working?

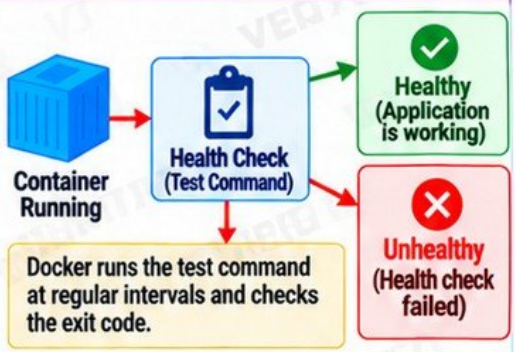
1. What Are Container Health Checks?

- A health check tests if the application inside the container is working.
- It detects unhealthy services, not just stopped containers.
- A container can be running but unhealthy if the application is not responding correctly.
- Health checks help dependent services know when a service is ready.

2. Container Health States

starting	Health check has not passed yet.
healthy	The application is working.
unhealthy	Health check failed after the configured number of retries.
none	No health check is configured.

3. How a Health Check Works (Diagram)



4. Health Check for PostgreSQL

compose.yaml (PostgreSQL service)

```yaml
services:
  db:
    image: postgres:16
    container_name: veriqta-db
    environment:
      POSTGRES_DB: veriqta
      POSTGRES_USER: veriqta
      POSTGRES_PASSWORD: veriqta123
    healthcheck:
      test:["CMD-SHELL", "pg_isready -U
        $$POSTGRES_USER -d $$POSTGRES_DB"]
      interval: 5s
      timeout: 5s
      retries: 10
```

- pg_isready checks if PostgreSQL is ready to accept connections.
- Docker marks the container as healthy only when the command succeeds.

5. Health Check for Flask Application

app/app.py (add a health endpoint)

```python
from flask import Flask, jsonify
app = Flask(__name__)

@app.route("/health")
def health():
    return jsonify({"status": "ok"}), 200
```

compose.yaml (API service)

```yaml
services:
  api:
    build: ./app
    container_name: veriqta-api
    ports:
      - "5000:5000"
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f",
      "http://localhost:5000/health"]
      interval: 10s
      timeout: 5s
      retries: 5
```

6. Health Check Options

Option	What It Does
healthcheck	Defines the health check for the container.
test	The command to run. Returns 0 if healthy.
interval	How often to run the test (e.g. 5s, 10s).
timeout	How long to wait for a response (e.g. 5s).
retries	Number of failures before marking the container as unhealthy.

7. Liveness vs Dependency Readiness

Liveness (Is it alive?)

- Checks if the application is running and responding (e.g. /health endpoint).
- Used for container health monitoring.

Dependency Readiness (Is it ready to be used?)

- Checks if dependeccies (e.g. database) are ready (e.g. pg_isready).
- Used with depends_on: condition: service_healthy.

8. Check the Health Status

docker compose ps

```
NAME          IMAGE          STATUS              PORTS
veriqta-api   veriqta-api    healthy (2s ago)    0.0.0.0:5000->5000/tcp
veriqta-db    postgres:16    healthy (10s ago)   5432/tcp
```

9. What Happens When a Health Check Fails?

- The container state becomes unhealthy.
- Docker still keeps the container running (unless you configure a restart policy).
- Dependent services using condition: service_healthy will not start until the service is healthy.
- An unhealthy container does not automatically restart with Docker Compose.

10. Troubleshooting Tips

- Check container health status: docker compose ps
- View health check logs: docker inspect <container_name>
- Check application logs: docker compose logs api
- Make sure the health endpoint returns HTTP 200.
- Verify the test command works inside the container.

11. Common Failure Scenarios

- Using the wrong health check command.
- Endpoint returns a non-200 status code.
- Service marked unhealthy due to slow startup.
- Health check depends on an external service that is not available.
- Assuming unhealthy containers automatically restart with Compose.

12. Key Takeaways

- Health checks verify application health, not just running containers.
- Use pg_isready for PostgreSQL.
- Add a /health endpoint for your application.
- Use condition: service_healthy for dependency startup.
- Always monitor health status and logs.

13. Docker Compose Logs and Troubleshooting

1. Why Logs Are Important?

- Logs show what the application actually reported.
- They help you understand failures and find the real cause.
- Restarting containers without investigating is poor engineering practice.
- Logs do not always show the root cause, but they give you valuable evidence to start with.

2. Common Troubleshooting Workflow

1. Check container status (docker compose ps -a)
2. Read logs (docker compose logs)
3. Follow live logs (docker compose logs -f)
4. Investigate inside the container (docker compose exec)
5. Fix the issue and verify it works

3. Useful Compose Commands

Command	What It Does
docker compose logs	Shows logs for all services.
docker compose logs -f	Follows logs in real time.
docker compose logs api	Shows logs for a specific service (e.g. api).
docker compose ps -a	Shows all containers including exited ones.
docker compose exec api sh	Runs a command inside a running container.
docker compose restart api	Restarts a specific service.

4. View All Service Logs

`docker compose logs`

```
veriqta-db  |  PostgreSQL Database directory
veriqta-db  |  ready to accept connections
veriqta-api |  Starting Flask application...
veriqta-api |  * Running on http://0.0.0.0:5000
veriqta-web |  /docker-entrypoint.sh:
veriqta-web |  Configuration complete
veriqta-web |  * Ready for connections
```

5. Follow Live Logs

`docker compose logs -f api`

```
veriqta-api | [2026-10-18 10:12:01]
INFO: Starting application...
veriqta-api | [2026-10-18 10:12:03]
ERROR: could not connect to
PostgreSQL: could not translate
host name "dbb" to address:
Name or service not known
veriqta-api | [2026-10-18 10:12:08]
INFO: Retrying connection...
veriqta-api | [2026-10-18 10:12:13]
ERROR: could not connect to
PostgreSQL...
```

6. Check Container Status

`docker compose ps -a`

```
NAME          IMAGE          COMMAND                  STATE      PORTS
veriqta-api   veriqta-api    "python app.py"          exited (1)
veriqta-db    postgres:16    "docker-entrypoint"      running    5432/tcp
veriqta-web   nginx          "nginx -g 'daemon..."    running    0.0.0.0:8080->80/tcp
```

- Exited containers can help you identify services that failed to start.

7. Run Diagnostic Commands Inside a Container

`docker compose exec api sh`

```
/app #  ping db
PING db (172.20.0.2): 56 data bytes
/app #  python -c "import os; print(os.getenv('DB_HOST'))"
dbb
```

8. Example Failure: Wrong Database Hostname

compose.yaml (api service with wrong hostname)

```
services:
  api:
    build: ./app
    environment:
      DB_HOST: dbb  # wrong hostname
      DB_NAME: veriqta
      DB_USER: veriqta
      DB_PASSWORD: veriqta123
    depends_on:
      - db
```

9. Observe the Failure in Logs

`docker compose logs api`

```
veriqta-api | ERROR: could not
connect to PostgreSQL: could not
translate host name "dbb" to address:
Name or service not known
veriqta-api | Retrying in 5 seconds..:
veriqta-api | ERROR: could not
connect to PostgreSQL...
```

10. Fix the Configuration and Verify Recovery

compose.yaml (correct hostname)

```
services:
  api:
    environment:
      DB_HOST: db  # correct service name
      DB_NAME: veriqta
      DB_USER: veriqta
      DB_PASSWORD: veriqta123
```

- Recreate the service: `docker compose up -d api`

Check the logs again to confirm the application connects successfully.

11. Common Issues and What to Look For

Issue	What to Check in Logs	Example Log Message
Database connection failure	Database hostname, port, credentials, network	could not translate host name "dbb" to address
Application crash	Python errors, missing environment variables	KeyError: 'DB_HOST'
Missing configuration	Check .env values and Compose interpolation	could not connect, invalid credentials
Port already in use	Check port mapping and host conflicts	bind: address already in use

12. Key Takeaways

- Use docker compose logs to investigate failures.
- Use docker compose logs -f to follow logs in real time.
- Use docker compose ps -a to find exited containers.
- Use docker compose exec to run diagnostic commands.
- Fix the real cause instead of restarting repeatedly.
- Logs tell you what the application reported, but they do not always reveal the root cause.

13. Junior Engineer Insight

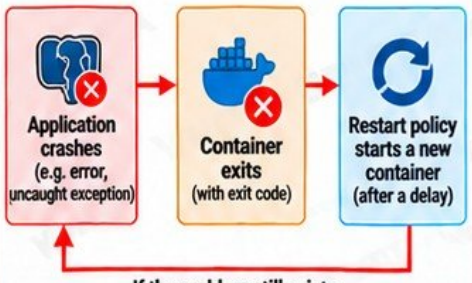
- A running container does not mean the application is working.
- Logs are your first source of evidence when things go wrong.
- Understand the error message and what part of the system is failing.
- Investigate step by step and verify your fix with logs.
- Good engineers solve the root cause, not just the symptom.

14. Restart Policies and Failure Recovery

1. Why Restart Policies Matter?

- Containers can stop unexpectedly due to crashes, errors or host issues.
- Restart policies tell Docker what to do when a container exits.
- They improve recovery from some failures.
- They do not fix application bugs or incorrect configuration.
- Automatic restarts can hide recurring problems if you don't investigate the root cause.

2. How Restart Policies Work (Diagram)



3. Restart Policy Options

Policy	What It Does
no	Do not restart the container (default).
always	Always restart, no matter how it stopped.
unless-stopped	Restart unless it was manually stopped.
on-failure	Restart only when the container exits with a non-zero exit code.
on-failure:N	Restart up to N times on failure (e.g. on-failure:5).

4. restart Policy in compose.yaml

compose.yaml

```yaml
services:
  api:
    build: ./app
    container_name: veriqta-api
    restart: unless-stopped  # try always,
                             # no or on-failure

  worker:
    image: my-worker
    restart: on-failure:5  # restart up to
                           5 times on failure
```

5. Container Exit Codes

- When a container stops, it returns an exit code.
- 0 → application exited successfully.
- Non-zero (1, 2, 137, etc.) → error or crash.
- You can see the exit code using:

```
docker compose ps -a
```

- Common exit codes:
  - 1 General application error
  - 137 Killed (e.g. out of memory)
  - 139 Segmentation fault
  - 143 Terminated (e.g. docker stop)

6. Application Crash vs Unhealthy

Application Crash (Container exits)

- The main process stops.
- Container exits with a non-zero exit code.
- Restart policy may start a new container.
- Example: Python application throws an exception and exits.

Unhealthy Container (Still running)

- Container is still running but health check fails.
- Marked as unhealthy.
- Restart policy does not act on health status.
- Example: Application is running but cannot connect to the database.

7. Demonstrate a Controlled Failure

1. Create a simple Flask app that exits with an error:

```
from flask import Flask
app = Flask(__name__)
raise Exception("Simulated crash")
```

2. Start the service:

```
docker compose up -d api
```

3. Watch the logs and restart behavior:

```
docker compose logs -f api
```

4. Check the restart count:

```
docker compose ps -a
```

8. Example Restart Loop in Logs

```
veriqta-api | Starting application...
veriqta-api | Traceback (most recent call last):
veriqta-api |   File "app.py", line 3, in <module>
veriqta-api |     raise Exception("Simulated crash")
veriqta-api | Exception: Simulated crash

Container veriqta-api exited with code 1
Restarting (1/5) in 5 seconds...

Container veriqta-api exited with code 1
Restarting (2/5) in 5 seconds...

Container veriqta-api exited with code 1
Restarting (3/5) in 5 seconds...
```

9. Check Restart Count

```
docker compose ps -a
```

NAME	STATE	RESTARTS
veriqta-api	Restarting (1)	3
veriqta-db	Up 2 minutes	0

10. Fix the Issue and Verify Recovery

1. Fix the code or configuration.
2. Recreate the service:

```
docker compose up -d api
```

3. Check that the container is now running:

```
docker compose ps
```

4. Confirm the logs show normal startup.

11. Common Operational Consequences

- Restart loops consume CPU and memory.
- Logs can be filled with repeated errors.
- It can hide real problems if you don't investigate.
- Other services may also be affected if they depend on the failing service.

12. Key Takeaways

- Use the right restart policy for your use case.
- Restart policies help recovery from some failures but cannot fix incorrect application configuration.
- Investigate repeated failures instead of relying only on automatic restarts.
- Monitor logs, exit codes and restart counts.
- Automatic restarts can hide recurring application problems.

13. Junior Engineer Insight

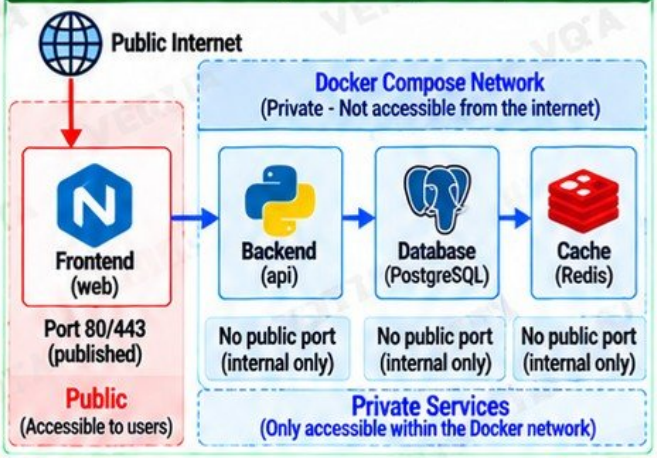
- Logs tell you what the application reported.
- Logs do not always reveal the root cause.
- A running container is not always a working application.
- Always investigate the actual error and fix the cause, not just restart the container.

15. Docker Compose Security Essentials

1. Why Security Matters?

- Containers are more secure when configured correctly.
- Avoid running containers as root unnecessarily.
- Reduce the risk of unauthorized access and lateral movement between services.
- Apply the principle of least privilege.
- Use secure configuration practices from the beginning.

2. Multi-Container Application (Security View)



3. Do Not Run as Root

compose.yaml

```yaml
services:
  api:
    build: ./app
    user: "1000:1000" # non-root
    # or use a non-root user
    # defined in the Dockerfile
```

- Running as root is not required for most applications.
- Use a non-root user to reduce potential damage if the container is compromised.

4. Keep Database Ports Private

compose.yaml (database service)

```yaml
services:
  db:
    image: postgres:16
    # Do NOT publish the port
    # ports:
    #   - "5432:5432"
    # Only accessible inside
    # the Docker network
```

- Do not publish PostgreSQL or Redis to the public internet.
- Only other containers in the same Docker network should access them.
- Publish a port only when strictly required (e.g. a public web service).

5. Compose Secrets (File-Based)

compose.yaml (using secrets)

```yaml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: veriqta
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password

secrets:
  db_password:
    file: ./secrets/db_password.txt
```

- Secrets are stored as files and mounted into containers.
- More secure than hardcoding passwords in the compose file.
- Use secrets for sensitive data such as database passwords, API keys, and tokens.

6. Image Version Pinning

compose.yaml

```yaml
services:
  db:
    image: postgres:16.4  # use a
                          # specific version
  redis:
    image: redis:7.2  # avoid 'latest'
  api:
    image: my-api:1.0.0
```

- Use specific image versions (e.g. postgres:16.4) instead of latest.
- Pull images from trusted sources (e.g. official images).
- Helps ensure consistent, predictable and more secure deployments.

7. Read-Only Filesystems and Capabilities

compose.yaml (additional security options)

```yaml
services:
  api:
    image: my-api:1.0.0
    read_only: true    # make filesystem read-only
    cap_drop:
      - ALL            # drop all Linux capabilities
    security_opt:
      - no-new-privileges:true
```

- Use read_only:true to prevent unnecessary writes inside the container.
- Drop unnecessary Linux capabilities.
- Use no-new-privileges to reduce privilege escalation risks.

8. Limits of Compose Secrets

- Compose secrets are useful for local development and simple deployments.
- They do not provide encryption at rest, automatic secret rotation, or access control.
- For production, use a dedicated secrets-management system (e.g. Docker Swarm, Kubernetes Secrets, AWS Secrets Manager, HashiCorp Vault).
- Do not commit real secrets to Git.

9. Security Best Practices

- Run containers as non-root users.
- Keep database and cache ports unpublished.
- Use Compose secrets for sensitive data.
- Pin image versions and use trusted images.
- Use read-only filesystems where possible.
- Drop unnecessary Linux capabilities.
- Follow the principle of least privilege.
- Regularly update images and dependencies.

10. Security Reminder

Never publish PostgreSQL or Redis to the public internet for convenience.

11. Common Mistakes

- Running containers as root.
- Publishing database ports to the internet.
- Using latest image tags.
- Hardcoding secrets in the compose file.
- Assuming Compose secrets are fully secure.
- Giving containers unnecessary privileges.

12. Key Takeaways

- Build containers with least privilege.
- Keep internal services private.
- Use secrets and secure configuration.
- Pin versions and use trusted images.
- Apply read-only filesystems and drop capabilities.
- Compose secrets do not replace a full production secrets-management system.

16. Development Workflows with Compose Watch

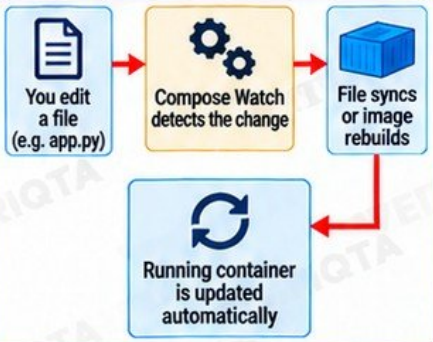
1. Why Use Compose Watch?

- Speeds up development by automatically applying changes.
- Reduces manual rebuild and restart steps.
- Keeps your development workflow simple and productive.
- Detects file changes and either syncs files or rebuilds the image.
- Helps you see changes in your running application quickly.

2. Rebuild vs Restart vs Watch

Action	What It Does	When to Use
Rebuild	Builds a new image from Dockerfile.	When dependencies or image files change (e.g. requirements.txt).
Restart	Stops and starts the existing container.	When configuration changes (e.g. env vars).
Watch	Automatically syncs changed files and rebuilds when needed.	During development to apply code changes quickly.

3. How Compose Watch Works (Diagram)



4. Using Compose Watch

Run with watch mode:

```
docker compose up --watch
```

```
$ docker compose up --watch
[+] Starting 2/2
 ✓ Container veriqta-api  Started
 ✓ Container veriqta-db   Started

Watching for file changes...
app.py changed, syncing...
Container veriqta-api  Recreated
veriqta-api  Started
```

- Compose Watch monitors your files.
- It syncs changed files or rebuilds the image based on your configuration.
- You see changes in your running application automatically.

5. File Synchronization and Rebuild Rules

Change Type	What Happens	Example
Source code change (e.g. app.py)	Files are synced to the running container.	Edit app.py → Instant update (no rebuild).
Template or static file change	Files are synced.	Edit templates/ index.html
Python dependency change (requirements.txt)	Image is rebuilt.	Add a new package → Triggers rebuild.
Dockerfile change	Image is rebuilt.	Edit Dockerfile → Triggers rebuild.
Configuration file change (compose.yaml)	Container is recreated.	Edit compose.yaml → Service recreated.

6. Example: Flask Source Change

1. Edit the API response in `app.py`:

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return {"message": "Hello from
    Compose Watch!"}
```

2. Save the file.
3. Compose Watch detects the change and syncs the file.
4. The running container is updated.
5. Test the change in your browser or with curl:

```
curl http://localhost:5000
```

Expected response:

```
{"message": "Hello from
 Compose Watch!"}
```

7. Compose Watch vs Bind Mounts

Feature	Compose Watch	Bind Mounts
How it works	Syncs files and rebuilds when needed.	Mounts your local files into the container.
Rebuilds image automatically	Yes (when required).	No.
Performance	Optimized for development.	Generally fast.
Best for	Modern development workflow with rebuild rules.	Simple local development.
Configuration	Uses watch configuration.	Uses volumes with host paths.

8. When Rebuilds Are Required

- When you change dependencies (e.g. requirements.txt).
- When you modify the Dockerfile.
- When you add system packages or build tools.
- When the application needs new files that must be included in the image.
- Code changes (e.g. app.py) are usually synced without a rebuild.

9. Keep Development Tools Separate

- Use Compose Watch only in your development configuration.
- Keep development-only tools and settings out of production.
- Do not include debug tools, hot-reload settings, or watch configuration in production compose files.
- Maintain separate files if needed (e.g. compose.yaml for production and compose.dev.yaml for development).

10. Practice: Make a Change and Verify

1. Start the stack with watch mode:
   `docker compose up --watch`
2. Edit the API response message in app.py.
3. Save the file and watch the logs.
4. Test the endpoint and confirm the new response.

11. Junior Engineer Tip

- Watch mode supports local development.
- It is not a production deployment or continuous delivery system.
- It helps you develop faster but you still need proper testing, building, and deployment for production.

12. Key Takeaways

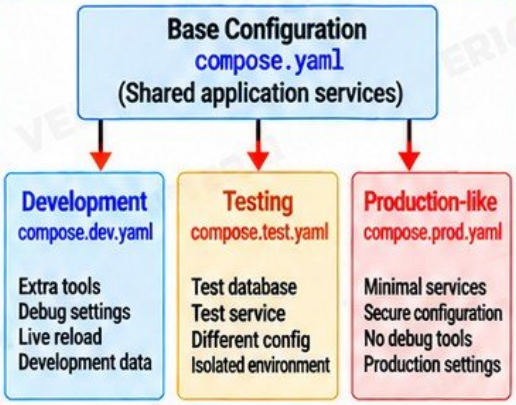
- `docker compose up --watch` speeds up development.
- It syncs files and rebuilds when needed.
- Use it for local development, not production.
- Know the difference between rebuild, restart, and watch.
- Dependency and Dockerfile changes require rebuilding the image.
- Keep development-only tooling out of production.

17. Multiple Environments and Compose Profiles

1. Why Multiple Environments?

- Different environments have different needs (development, testing, production-like).
- Each environment may use different services, configuration, and settings.
- Helps keep development tools out of production.
- Makes the application safer, more reliable and easier to manage.

2. Environment Structure (Diagram)



3. Using Multiple Compose Files

Basic usage (single file)

```
docker compose up
```

Using multiple files (override)

```
docker compose -f compose.yaml \
    -f compose.dev.yaml up
```

Specific environment examples

```
# Development environment
docker compose -f compose.yaml \
    -f compose.dev.yaml up
# Testing environment
docker compose -f compose.yaml \
    -f compose.test.yaml up
# Production-like environment
docker compose -f compose.yaml \
    -f compose.prod.yaml up -d
```

4. Compose Profiles for Optional Services

compose.yaml (using profiles)

```
services:
  api:
    build: ./app
    profiles:
      - core
  db:
    image: postgres:16
    profiles:
      - core
  test:
    image: curlimages/curl
    command:["curl", "http://api:5000/health"]
    profiles:
      - test
  admin:
    image: adminer:latest
    ports: - "8080:8080"
    profiles: - dev
```

Run with a specific profile

Start core services only (default)

```
docker compose up
```

Start with test profile

```
docker compose
--profile test up
```

Start with dev profile

```
docker compose
--profile dev up
```

Start multiple profiles

```
docker compose
--profile dev --profile
test up
```

5. Example: Development Override File

compose.dev.yaml

```
services:
  api:
    volumes:
      - ./app:/app
    environment:
      FLASK_DEBUG: "1"
    profiles:
      - dev
  adminer:
    image: adminer:latest
    ports:
      - "8080:8080"
    profiles:
      - dev
```

What this does:

- Adds a bind mount for live development.
- Enables debug mode for development only.
- Adds Adminer for database access (only in dev).
- This file overrides or extends the base compose.yaml.

6. Validate the Final Configuration

- The docker compose config command shows the final merged configuration.
- Always review the output to make sure the correct services and settings are used.

```
docker compose -f compose.yaml \
    -f compose.dev.yaml config
```

7. Example: Test Service (Runs Only When Requested)

compose.test.yaml

```
services:
  test:
    image: curlimages/curl
    command: ["curl", "-f", "http://api:5000/health"]
    profiles:
      - test
  api:
    environment:
      ENV: "test"
    profiles:
```

Run the test service

```
docker compose
  --profile test up
```

- The test service runs only when the test profile is used.
- Useful for running automated checks or manual tests.

8. Configuration Review Checklist

- ✓ Use the correct compose files for each environment.
- ✓ Review environment variables and secrets.
- ✓ Check that debug tools are not enabled in production.
- ✓ Validate the final configuration with docker compose config.
- ✓ Ensure only required services are running.
- ✓ Confirm that ports are correctly configured.

9. Common Mistakes

- ✗ Accidentally enabling debug mode in a public environment.
- ✗ Leaving development tools (e.g. Adminer) running in production.
- ✗ Using the wrong compose file.
- ✗ Exposing internal services to the internet.
- ✗ Not reviewing the final configuration.
- ✗ Forgetting to disable or remove test services.

10. Key Takeaways

- ✓ Use multiple compose files to manage different environments.
- ✓ Use profiles for optional services.
- ✓ Keep development and testing tools out of production.
- ✓ Always review environment-specific configuration.
- ✓ Validate the final configuration before running.

11. Security Reminder

Do not accidentally enable debugging services, development settings, or internal tools in a public environment.

12. Junior Engineer Insight

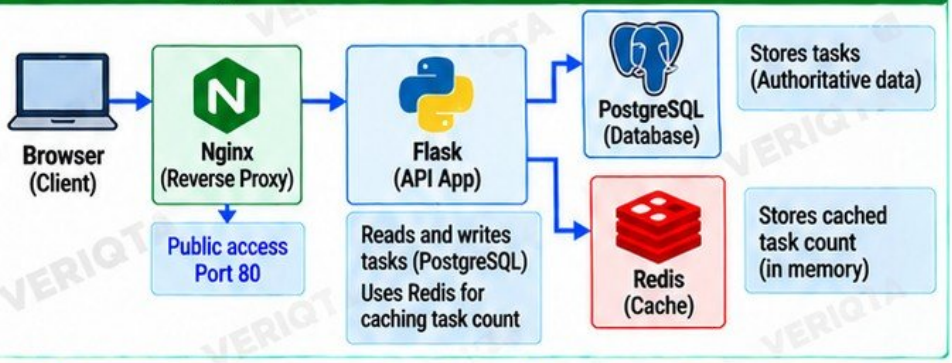
- Compose profiles give you flexibility without maintaining multiple completely separate files.
- Always understand which services are running and why.
- Review the final configuration to avoid surprises.

18. Add Redis and Complete the Four-Service Stack

1. What Is Redis and Why Use It?

- Redis is an in-memory data store used for caching, session storage, rate limiting and more.
- It makes applications faster by storing frequently accessed data in memory.
- Caching reduces database load and improves response time.
- In this stack, Redis caches the task count while PostgreSQL remains the authoritative source of data.

2. Four-Service Application Architecture



3. Add Redis to docker-compose.yml

docker-compose.yml (four services)

```
services:
  nginx:
    image: nginx:latest
    ports:
      - "80:80"
    depends_on:
      - api
  api:
    build: ./app
    environment:
      DATABASE_URL: postgresql://veriqta:veriqta123@db:5432/tasks
      REDIS_URL: redis://redis:6379/0
    depends_on:
      - db
      - redis
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: veriqta
      POSTGRES_PASSWORD: veriqta6379
      POSTGRES_DB: tasks
    volumes:
      - db_data:/var/lib/postgresql/data
  redis:
    image: redis:7.2
    command: ["redis-server", "--appendonly", "yes"]
    volumes: - redis_data:/data
volumes:
    db_data:
    redis_data:
```

4. Connect Flask to Redis

app.py (simplified example)

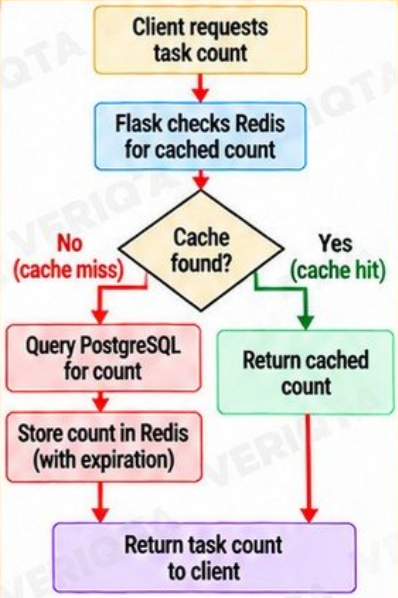
```
from flask import Flask, jsonify
from redis import Redis
from sqlalchemy import create_engine, text

app = Flask(__name__)
# Redis connection using service name
redis_client = Redis(host="redis", port=6379, db=0)
# PostgreSQL connection
engine = create_engine(
    "postgresql://veriqta:veriqta123@db:5432/tasks"
)

@app.get("/tasks/count")
def get_count():
    cached = redis_client.get("task_count")
    if cached:
        return jsonify({
            "count": int(cached),
            "cache": "hit"
        })
    with engine.connect() as conn:
     result = conn.execute(text("SELECT COUNT(*) FROM tasks"))
     count = result.scalar()

    # Cache for 60 seconds
    redis_client.setex("task_count", 60, count)
    return jsonify({
          "count": count,
          "cache": "miss"
    })
```

5. Cache Flow (Diagram)



6. Cache Hits, Misses and Expiration

- Cache hit: Data is found in Redis (faster response).
- Cache miss: Data is not found, so the app queries PostgreSQL, then stores the result in Redis.
- Expiration: Cached data automatically expires after a set time (e.g. 60 seconds).

7. Invalidate Cache After Task Updates

```
@app.post("/tasks")
def create_task():
    # save task to PostgreSQL
    ...
    # Invalidate cached task count
    redis_client.delete("task_count")
 return jsonify({"message": "Task created"})
```

8. What Happens If Redis Is Unavailable?

- The application should still work.
- Caching will fail, but the app can fall back to querying PostgreSQL.
- Response time may be slower.
- Handle Redis connection errors in the code.
- Do not store critical or only copies of data in Redis.

9. Practice: Test the Four-Service Stack

1. Start the stack: `docker compose up -d`
2. Create a task using the API (POST /tasks).
3. Retrieve the task count multiple times and check for cache hit and miss in the response.
4. Create another task and verify that the cached count is invalidated and updated.

10. Production Awareness

- Use Redis as a cache, not as the only place for important application data.
- PostgreSQL remains the authoritative data store.
- Configure memory limits, persistence and monitoring for Redis in production.
- A cache can improve performance, but should never silently become the only source of data.

11. Key Takeaways

- Redis adds a real caching layer to the app.
- Flask connects to Redis using its service name.
- Cache hits are faster, cache misses query PostgreSQL.
- Invalidate cached data after updates.
- Handle Redis failures gracefully.
- A cache is not a replacement for the database.

12. Common Mistake

Accidentally enabling debugging services, or relying on Redis as the only place for important application data.

13. Junior Engineer Insight

- Caching improves performance but adds complexity.
- Always understand what is cached, when it expires, and how it is invalidated.
- Monitor your cache and database in production.

19. Break the Application and Recover It

1. Scenario: What's Happening?

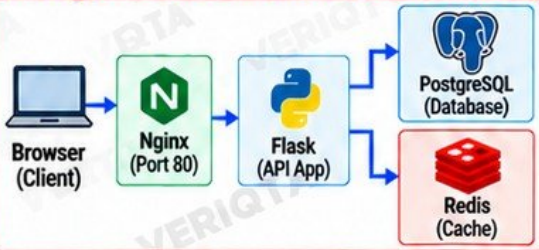
The VERIQTA Task App was working yesterday, Today, the frontend loads, but creating tasks fails.

- The application has four services: Nginx, Flask, PostgreSQL and Redis.
- You will investigate four controlled failures and restore the application.
- Use logs, container status, and diagnostic commands to find the cause and apply the smallest safe correction.
- Do not delete PostgreSQL data while fixing the issues.

2. Troubleshooting Process (Follow This)

1. **Identify the visible symptom** — What is the user seeing?
2. **Inspect container status** — `docker compose ps`
3. **Read relevant logs** — `docker compose logs <service>`
4. **Form a hypothesis** — What might be the cause?
5. **Run a diagnostic command** — Check and confirm.
6. **Apply the smallest safe correction** — Fix only what is needed.
7. **Verify the application** — Test that it works again.

3. Application Architecture (Four Services)



4. docker compose ps (Example)

```
$ docker compose ps
```

NAME	STATE	PORTS
veriqta-nginx	running	0.0.0.0:80->80/tcp
veriqta-api	running	5000/tcp
veriqta-db	running	5432/tcp
veriqta-redis	running	6379/tcp

5. Failure 1: Wrong Database Hostname

Symptom

Creating a task fails. Error in the API.

Evidence (Logs)

```
$ docker compose logs api
dial tcp: lookup postgress
on 127.0.0.11:53:
no such host
```

Cause

The API is using the wrong database hostname (e.g. postgress instead of db).

Diagnostic Command

Check the environment variable:

```
$ docker compose exec api \
  env | grep DATABASE_URL
```

Fix

Update the database hostname to db in compose.yaml or .env. Recreate the API service.

Verify

Create a new task. It works and is saved in PostgreSQL.

6. Failure 2: Database Not Ready When API Starts

Symptom

API crashes on startup. Creating tasks fails.

Evidence (Logs)

```
$ docker compose logs api
psycopg2.OperationalError:
connection to server at "db" (172.19.0.2),
port 5432 failed: Connection refused
```

Cause

The API starts before PostgreSQL is ready to accept connections.

Diagnostic Command

Check database logs:

```
$ docker compose logs db
```

Fix

Add a health check to the database and use depends_on with condition: service_healthy. Recreate the stack.

Verify

API connects successfully and you can create tasks.

7. Failure 3: Host Port Already Occupied

Symptom

Nginx fails to start. The frontend is not accessible.

Evidence (Logs)

```
$ docker compose logs nginx
Error starting userland proxy:
Bind for 0.0.0.0:80 failed:
port is already allocated
```

Cause

Port 80 on the host machine is already in use by application.

Diagnostic Command

Find what is using the port:

```
$ sudo lsof -i :80
# or
$ sudo netstat -tulnp | grep :80
```

Fix

Change the host port in compose.yaml (e.g. 8080:80) or stop the conflicting service.

Verify

Access the app at http://localhost:8080 and confirm it loads and you can create tasks.

8. Failure 4: Redis Unavailable

Symptom

Creating tasks is slow or fails. API shows Redis connection errors.

Evidence (Logs)

```
$ docker compose logs api
redis.exceptions.ConnectionError:
Error 111 connecting to redis:6379.
Connection refused.
```

Cause

Redis container is not running or not reachable by the API.

Diagnostic Command

Check Redis status and logs:

```
$ docker compose ps redis
$ docker compose logs redis
```

Fix

Start the Redis service or correct the Redis hostname/port. Recreate the API service if needed.

Verify

Create a task and check that the task count is cached and returned correctly.

9. Practice: Restore the Complete Application

1. Fix each issue one by one.
2. Verify all four services are running.
3. Create a new task and confirm it is saved in PostgreSQL.
4. Check that the task count is cached in Redis.
5. Ensure the frontend, API, database and cache all work together.
6. Do not delete PostgreSQL data during the recovery process.

10. Common Mistakes

- ✗ Assuming docker compose up means everything is working.
- ✗ Not checking logs when something fails.
- ✗ Using the wrong service name.
- ✗ Ignoring container health and startup order.
- ✗ Leaving host ports in use.
- 6 Not verifying that the application features work from the user's perspective.

11. Junior Engineer Insight

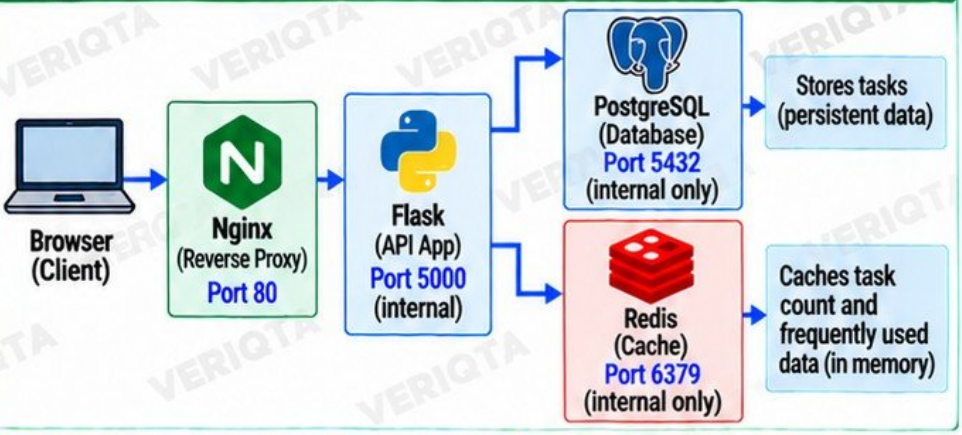
- A successful `docker compose up` does not prove that the application's user-facing features work.
- Always test the actual functionality (after fixing the issue) from the user's point of view.

20. Final Project: Run, Verify, and Hand Over the Application

1. Project Overview

- You will complete a real four-service application and demonstrate that it works end to end.
- Build a custom Flask image.
- Use Docker Compose to run Nginx, Flask, PostgreSQL and Redis.
- Verify all required features.
- Troubleshoot a failure and recover it.
- Document clear instructions in a README for your GitHub portfolio.

2. Application Architecture (Four Services)



3. Build the Flask Image

Build the image using Dockerfile

```
$ docker compose build api
[+] Building 10.4s (8/8)
 => [internal] load build definition
 => [api] building image veriqta-api
 => => naming to veriqta-api:latest
Successfully built veriqta-api
```

4. Start the Application

Start all services with Docker Compose

```
$ docker compose up -d
[+] Running 4/4
 ✓ Container veriqta-db     Started
 ✓ Container veriqta-redis  Started
 ✓ Container veriqta-api    Started
 ✓ Container veriqta-nginx  Started
```

5. Validate the Configuration

Check the final merged configuration

```
$ docker compose config
# Review the output to confirm
# services, ports, environment
# variables, networks and volumes
# are correct.
```

6. Access the Application

Open the frontend in your browser



7. Test the API

Create and retrieve tasks using the API

```
$ curl -X POST http://localhost:5000/tasks \
   -H "Content-Type: application/json" \
   -d '{"title": "New Task"}'
$ curl http://localhost:5000/tasks
{
  "tasks": [
    { "id": 1, "title": "New Task" }
  ]
}
```

8. Verify PostgreSQL Persistence

Recreate containers and confirm data remains

```
$ docker compose down
$ docker compose up -d
# Access the app and confirm
# your tasks are still there.
# The PostgreSQL data is stored
# in a persistent volume.
```

9. Verify Redis Cache Behavior

Check cache hits and misses

```
$ curl http://localhost:5000/tasks/count
{ "count": 2, "cache": "miss" }
$ curl http://localhost:5000/tasks/count
{ "count": 2, "cache": "hit" }
# Create a new task and check again
# The cache should be invalidated.
$ curl -X POST http://localhost:5000/tasks ...
$ curl http://localhost:5000/tasks/count
{ "count": 3, "cache": "miss" }
```

10. Check Health and Logs

Verify service health and inspect logs

```
$ docker compose ps
$ docker compose logs -f api
$ docker compose logs db
$ docker compose logs redis
$ docker compose exec api curl localhost:5000/health
```

11. Recover from a Failure

Introduce a failure and fix it

1. Intentionally break the application (e.g. wrong database hostname).
2. Observe the error and check logs.
3. Identify the cause.
4. Apply the smallest safe correction.
5. Verify the application works again.

12. Stop and Clean Up Safely

Remove containers without deleting data

```
$ docker compose down
# This removes the containers but
# keeps the PostgreSQL volume and data.
```

13. Document the Project

Create a README with clear instructions

- Project overview and architecture.
- How to build and start the application.
- How to verify the application works.
- Troubleshooting guide.
- How to stop and clean up (without deleting data).
- Screenshots or example outputs.

14. Final Deliverable

A working, reproducible Docker Compose project with a clear README, suitable for a junior engineer's GitHub portfolio.

15. Final Engineering Lesson

Deployment is not complete when containers start. It is complete when the application's required behavior has been verified.